

Algoritmo: metodo automatizzato per risolvere un problema

Focus corso

1) Progettazione algoritmo (Disegnazione algoritmo) $\longrightarrow$ tecniche di progettazione
(ricorsione)

$\downarrow$
Divide
et
impera

2) Analisi $\longrightarrow$ correttezza $\longrightarrow$ test
 $\downarrow$
efficienza $\longrightarrow$ tempo
 $\searrow$
spazio
 $\searrow$
dimostrabilità

ordinamento

Algoritmi di ordinamento numerico

- molto semplice
- facile da dimostrare

Strutture Dati: Metodo per risolvere in modo più efficiente dati ed operazioni

Array, Liste, Pile, Code, Alberi, ABR (Alberi Binari di Ricerca), Binary Heap, Grafi

Heap

Cos'è un heap?: L'heap è una struttura dati che ha proprietà simili a quelle di un albero binario di ricerca, tutto ciò contenuto all'interno di un array

Proprietà dell'heap: A differenza degli ABR, l'heap non "ragiona" e "figli" ma "ragione" e livelli, abbiamo 2 tipologie di heap:

1) Max-heap - La radice dell'albero è il suo massimo, e tra un livello e l'altro quello che contiene gli elementi di valore superiore è il livello più alto (Questo è il tipo che abbiamo trattato per la maggior parte) e lezione

2) Min-heap - La radice dell'albero è il suo minimo, e tra un livello e l'altro quello che contiene gli elementi di valore superiore è il livello più basso

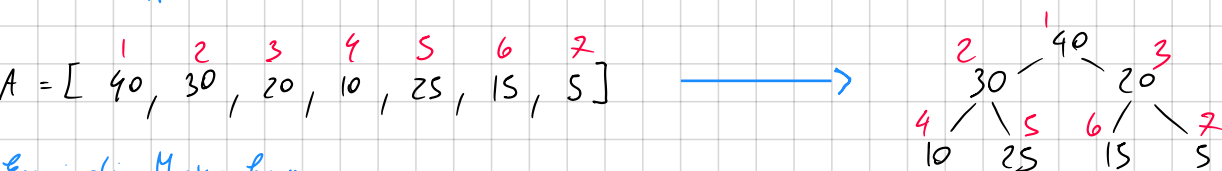
Rappresentazione di un heap: All'interno di un heap (sia max che min) il primo elemento dell'array è la radice

Come trovo il figlio x di un nodo?: $2 \cdot \text{posizione nodo}$

Come trovo il figlio dx di un nodo?: $(2 \cdot \text{posizione nodo}) + 1$

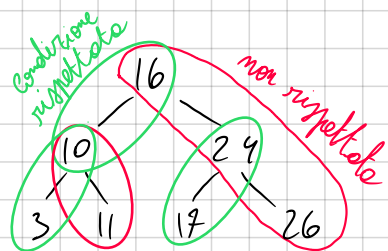
Come trovo il parent di un nodo?: $\text{posizione nodo} / 2$

Esempio rappresentazione

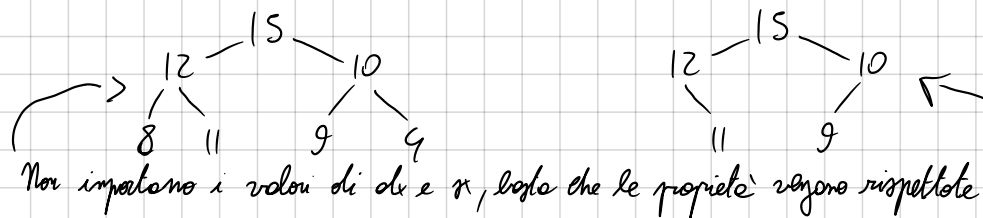


Esempi di Max-heap

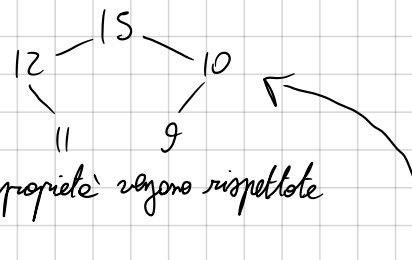
1) Questo non è un heap ma un ABR



2) Questo è un heap



3) Questo non è un heap



Basta anche solo un caso di condizione non rispettata per far sì che non sia un heap

Non è un albero completo

A cosa serve un heap? : 1) Code con queue (Lo vediamo dopo)

2) Algoritmo di ordinamento HeapSort (Lo vediamo tra poco)

Come costruisco un heap partendo da un array?

Ho bisogno di 2 metodi, uno che mi "ristemi" un determinato sottoalbero in modo tale che rispetti le proprietà, ed un secondo che mi applichi il 1° metodo all'intero albero

1° metodo - Max Heapify

Cos'è? : È una procedura che serve a ripristinare la struttura di max-heap partendo da un nodo che potrebbe violare

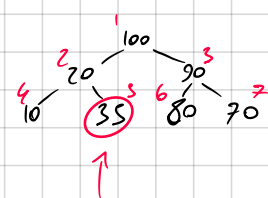
Come funziona? : Dato un nodo i assume che i sotto-alberi sinistro e destro di i siano già max-heap ma che il nodo i potrebbe non essere maggiore dei suoi figli.

La procedura è:

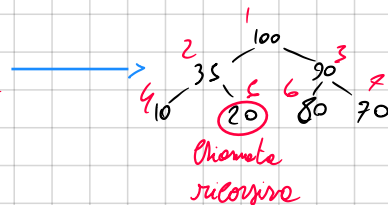
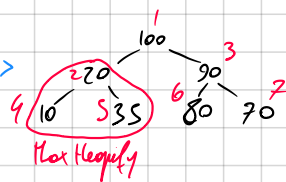
- 1) Confronta i con i suoi figli
- 2) Se uno dei figli è più grande di i , scambia i con il figlio più grande
- 3) Ripete ricorsivamente il processo sul figlio con cui è stato fatto lo scambio

Esempio

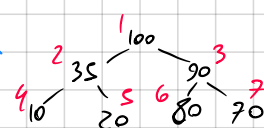
$A = [100, 20, 90, 10, 35, 80, 70]$



$35 > 20$
la proprietà non è rispettata



$A = [100, 35, 90, 10, 20, 80, 70]$



Proprietà
rispettata

procedure MaxHeapify (A, i)

// Calcolo i figli di i

$l = 2 * i$
 $r = (2 * i) + 1$
 $largest = i$

$\Theta(1)$

// Controllo i figli

if $l < \text{heapsize}$ and $A[l] > A[largest]$

$largest = l$

$\Theta(1)$

if $r < \text{heapsize}$ and $A[r] > A[largest]$

$largest = r$

// Se uno dei figli è più grande del nodo corrente con il maggiore tra i figli

if $largest \neq i$

$temp = A[i]$

$A[i] = A[largest]$

$A[largest] = temp$

Caso migliore $\rightarrow i = largest \rightarrow \Theta(1)$

Caso peggiore $\rightarrow i \neq largest \rightarrow T(n) = \Theta(1) + T(n-1) = \Theta(n)$
 n è l'altezza del nodo corrente

// Chiama ricorsivamente per assicurarsi che il sotto-albero sia ancora un heap
MaxHeapify (A, largest)

Tempi di calcolo

Caso migliore $\Theta(1)$: Questo perché nel caso migliore, ossia quando il sottoalbero è appunto
effettivo dei semplici controlli "senza fare niente"

Caso peggiore $\Theta(n)$: Due o tre volte delle chiamate ricorsive, quindi il tempo
impiegato dipende dall'altezza e cui sta

2° metodo - build Max Heap

Cos'è? È una procedura che trasforma un array qualsiasi in un max-heap

Come funziona: Applica iterativamente maxHeapify ad ogni nodo interno partendo dalle foglie

procedura build Max Heap

Tempi di calcolo: $\Theta(n)$

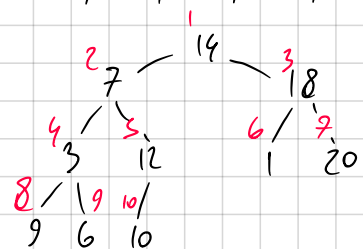
heapSize = A.length

for $i = (\text{heapSize} / 2)$ down to 1

 maxHeapify(A, i)

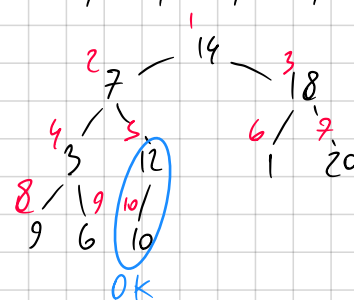
Esempio

A = [14, 7, 18, 3, 12, 1, 20, 9, 6, 8] $\Rightarrow$ heapSize = 10



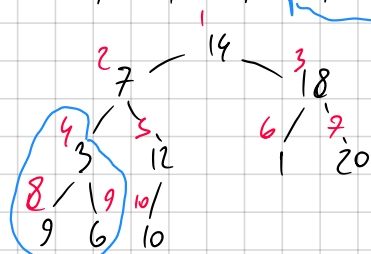
1) Heapify (A, 5)

A = [14, 7, 18, 3, 12, 1, 20, 9, 6, 8]



3) Heapify (A, 4)

A = [14, 7, 18, 3, 12, 1, 20, 9, 6, 8]

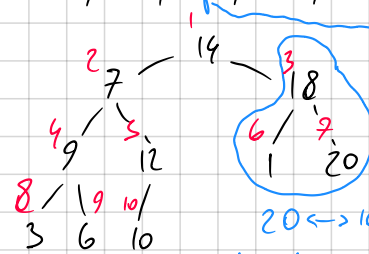


3 $\leftrightarrow$ 9

e chiamate ricorsiva su posizione 8

4) Heapify (A, 3)

A = [14, 7, 18, 9, 12, 1, 20, 3, 6, 8]

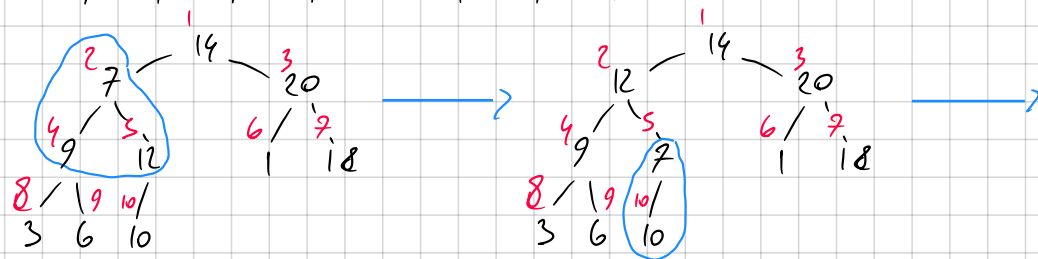


20 $\leftrightarrow$ 18

chiamate ricorsiva su posizione 7

5) Heapify (A, 2)

A = [14, 7, 20, 9, 12, 1, 18, 3, 6, 8]



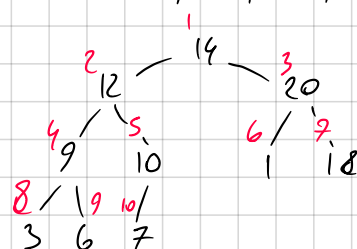
7 $\leftrightarrow$ 12

e chiamate ricorsivo su nodo 5

10 $\leftrightarrow$ 7

chiamate ricorsivo su nodo 10

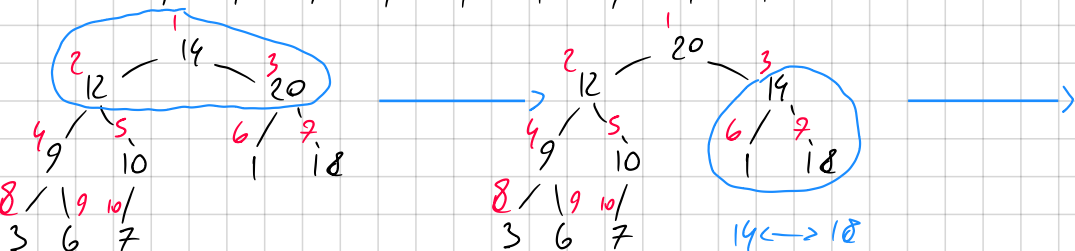
A = [14, 12, 20, 9, 10, 1, 18, 3, 6, 7]



miao :3

6) Heapify (A, 1)

A = [14, 12, 20, 9, 10, 1, 18, 3, 6, 7]



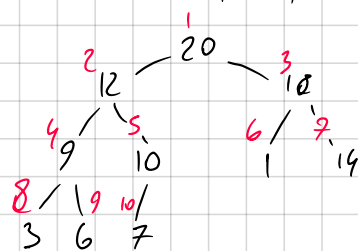
14 $\leftrightarrow$ 18

chiamate ricorsivo al nodo 7

14 $\leftrightarrow$ 20

chiamate ricorsivo sul nodo 3

A = [20, 12, 18, 9, 10, 6, 14, 3, 6, 7]



Heap Sort

Cos'è? : heapSort è un algoritmo di ordinamento deterministico, in-place, non stabile e con complessità temporale garantita di $\Theta(n \lg n)$

Come funziona? : 1) Prendo l'array disordinato e lo trasformo in un heap

2) Una volta costruito l'heap :

1) Scambio l'elemento alla radice (massimo) con l'ultimo

2) Riduco la dimensione dell'heap di 1 (escludendo l'elemento "più ordinato")

3) Applico maxHeapify alla radice per ripristinare la proprietà di max-heap

È in-place? : Sì, non utilizzo memoria aggiuntiva oltre quella dell'array dato

È stabile? : No, perché durante gli scambi e la ristrutturazione dell'heap, l'ordine originale tra elementi uguali non viene mantenuto

Esempio

$A = [4, 10, 3, 5, 1]$ $\xrightarrow{\text{build Max Heap}(A)}$ $A = [10, 5, 3, 4, 1]$

2) Ordinamento e scambio

$A = [10, 5, 3, 4, 1]$ $\xrightarrow{\text{heapsize}--}$ $A = [1, 5, 3, 4]$ $10] \xrightarrow{\text{build Heap}}$ $A = [5, 4, 3, 1]$ $10]$

Scambio $A = [1, 4, 3, 5]$ $10] \xrightarrow{\text{heapsize}--}$ $A = [1, 4, 3]$ $5, 10]$ ripeto finché non arrivo a : $A = [1, 3, 4, 5, 10]$

Procedure heapSort

tempi di calcolo : $\Theta(n \lg n)$ in ogni caso

```
A = buildMaxHeap(A)
for i = heapsize down to 1
    scambio A[1] con A[i]
    heapsize --
    maxHeapify(A, 1)
```

Code con priorità

Così sono? : Le code con priorità sono strutture dati che gestiscono elementi con specifiche una priorità e permettono di:

- Inserire un elemento con una priorità
- Estrarre l'elemento con priorità massima (o minima in caso di Min-Heap)

Come funzionano? : Le implementiamo attraverso l'utilizzo di un Max-Heap insieme a delle funzionalità aggiuntive che sono:

1) Estrai massimo

procedura $\text{HeapExtractMax}(S)$

```
// Controlla underflow
if heapsize == 0 then error "underflow"

// Prende il massimo e lo mette in code
max = S[1]
scambia S[1] e S[heapsize]

// Ripristina la struttura dell'heap
heapsize --
MaxHeapify(A, 1)  $\square \Theta(\log n)$ 

// Output
return max  $\square \Theta(1)$ 
```

2) Incrementa priorità

procedura $\text{HeapIncreaseKey}(S, x, k)$

```
// preconditione
if S[x] > k then error "Key is smaller"

// Aggiorna il valore alla nuova chiave più grande
S[x] = k

// Finché non siamo alla radice e la chiave è maggiore del padre spostiamo in alto
while x > 1 and S[parent(x)] < S[x] do
    scambia S[parent(x)] con S[x]
    x = parent(x)  $\square \Theta(\log n)$ 
```

3) Incremento - Insert(x) :

procedura $\text{insert}(S, x)$

```
// Controlla overflow
if S.length == S.heapsize then error "overflow"

// incrementa la grandezza dell'heap
S.heapsize = S.heapsize + 1

// Inserisce un valore di placeholder
S.heapsize = -∞

// Posiziona correttamente il nuovo valore
HeapIncreaseKey(A, A.heapsize, key)
```